filter’s false positive ratio, so we need to accurately count the elements actually stored. Notice how we pass the positions array as second argument to `contains`: as mentioned in the previous section, this allows us to avoid computing it again in `contains`, and so to perform this expensive operation only once for each call to `insert`.

#4 For each index, we need to write a `1` in our buffer (here, using functional notation, we can leverage map, just ignoring its result, or use a more appropriate functional operator like `reduce` or `forEach`).

This is because the precision of the filter would not be altered by adding twice, trice or an infinite number of times the same key: to all extents, it would count as if the key was added once.

There is a twist, though: if we add a new unique key `x` for which all bits’ indices clashes with locations already set to `1`, it will be treated as a duplicate key and the data strucure’s size won't be incremented. Again, this makes perfect sense if you think that, in that situation and before actually adding the new colliding key, a call to `contains(x)` would have returned a false positive anyway.